

Hands-On-Learning: Backend Reliability Challenge - Handle It Smart

Learning Objectives

By the end of this activity, learners will be able to:

- Implement a lightweight middleware service that wraps LLM API calls behind a stable endpoint.
- Handle timeouts and rate limits using structured retries and exponential backoff.
- Add basic caching to avoid redundant LLM requests.
- Log token usage and key metrics to make performance and cost visible.
- Validate backend behavior using Postman under simple “stress-like” conditions.



Description

In this hands-on practice, you will act as the backend developer responsible for making an LLM-powered chat assistant survive real traffic. You'll build a small middleware service that wraps the OpenAI API, adds retries and caching, and logs token usage so your application remains responsive and predictable even when the provider is slow or rate-limited.

Structure of the Activity

Title: Hands-on Practice: Backend Reliability Challenge - Handle It Smart

Dataset information / Resources Required:

→ No dataset required; you will work with live or test OpenAI API calls.

→ Sample prompt and simple chat payloads (e.g., user messages) that simulate typical chat requests.

Tools used:

- **Backend:** FastAPI (Python) or Node.js (Express/Nest)
- **LLM Provider:** OpenAI API
- **Testing:** Postman (or any other HTTP client like Insomnia, curl)

Setup Requirements:

- OpenAI API key configured via environment variable or config file.
- Python + FastAPI + Uvicorn or Node.js + Express/Nest installed
- Ability to run a local server (e.g., `uvicorn main:app --reload` or `node server.js`).
- Postman installed or access to a similar API client.

Scenario

You are a backend developer for an AI startup preparing to launch a chat assistant.

The prototype works on your machine, but during testing it:

- Hits rate limits (HTTP 429) and drops requests.
- Times out randomly because of slow network or long model reasoning.
- Exposes raw provider errors to the frontend.
- Gives no insight into how many tokens are being consumed.

Stakeholders are worried that even a modest traffic spike will break user sessions and make the product look unreliable. Your job is to build a **lightweight middleware service** that stabilizes all LLM calls: handling rate limits, retrying transient failures, caching repeated requests, and logging token usage for cost awareness.

Objective

Build and test a resilient backend endpoint that wraps an OpenAI chat completion call with retries, caching, and token logging, then demonstrate its behavior with sample requests.



Instructions for Participants

Getting Started

- 1 Choose your stack: FastAPI + Python or Node.js + Express/Nest.
- 2 Create a new project folder (e.g., llm_middleware_lab/).
- 3 Configure your OpenAI API key securely (environment variable or config file, not hard-coded).
- 4 Create a basic server skeleton with a single POST endpoint (e.g., /api/chat).

Activity 1: Build the Reliable Middleware Wrapper

Objective

Implement a backend function that calls the OpenAI API with retries, exponential backoff, optional caching, and token usage logging.



Input/Data:

Incoming JSON payload from the frontend, e.g.:

```
{  
  "user_message": "Can I change my subscription plan next month?"  
}
```

Simple static system prompt (e.g., “You are a helpful support assistant...”).

Steps to Complete

Step 1: Define a Wrapper Function

Create a function such as `callLLMWithRetry()` that:

- Accepts a message or prompt payload.
- Calls the OpenAI API (chat or completions).
- Wraps the call in a try/except (Python) or try/catch (Node.js).

Step 2: Implement Timeout & Retry Logic

- Set a **request timeout** (e.g., via HTTP client or OpenAI client options).
- Add **exponential backoff** for transient failures (especially 429 and typical 5xx errors), such as:
 - Retry up to 2-3 times.
 - Wait 0.5s, then 1s, then 2s between attempts.
- Ensure you **do not** retry on invalid input errors (4xx like 400).

Step 3: Add Simple Caching

- Use an in-memory cache (Python dict / Node Map) keyed by:
 - A hash of the user message (+ system prompt if you like).
- For repeated requests with the same key:
 - Return the cached response instead of calling the API again.
- Optionally, implement a simple TTL (time-to-live) if you have time.

Step 4: Log Token Usage and Metadata

→ Extract token usage from the OpenAI response if available (e.g., `usage.prompt_tokens`, `usage.completion_tokens`).

→ Log to console or a simple log structure:

→ Timestamp

→ Number of retries used

→ Token totals per request

→ Make logs human-readable (e.g., "[2025-11-23] /api/chat - retries=1, tokens=120 (prompt=70, completion=50)").

Step 5: Expose a Clean API Response

Design your endpoint to return a stable JSON shape, e.g.:

```
{  
  "answer": "...",  
  "meta": {  
    "retried": true,  
    "retry_count": 1,  
    "prompt_tokens": 70,  
    "completion_tokens": 50  
  }  
}
```

Ensure that if all retries fail, the endpoint returns a **user-friendly error message** and not a raw provider error.

Expected Outcome

A working /api/chat endpoint that:

- Retries on transient failures with backoff.
- Caches repeated requests.
- Logs token usage and retry counts.
- Returns stable, frontend-friendly JSON.

Tips

Keep your wrapper function small and focused; you can always refactor later.

Start with a fixed model and temperature—don't optimize everything at once.

Activity 2: Test, Observe, and Tweak Behavior

Objective

Use Postman to test the middleware under simple “stress-like” conditions and observe logs for reliability and cost patterns.



Input/Data:

- The running backend service on `http://localhost:<port>/api/chat`.
- A few sample user messages (including repeated ones).

Steps to Complete

Step 1: Basic Single Request Test

In Postman, create a POST request to `/api/chat` with a JSON body containing `user_message`.

Send the request and verify:

- Response structure matches your design.
- Answer string looks reasonable.
- Logs show token usage and retry count.

Step 2: Test Repeated Requests (Caching Check)

Send the **same** request multiple times.

Confirm that:

The second/third request either **hits cache** (visible from logs or added metadata) or at least does not trigger unnecessary retries.

Response times improve for cached calls (you can eyeball this from Postman timing and logs).

Step 3: Simulate Stress / Error Conditions (If Feasible)

Optionally, temporarily lower rate limits (if you simulate them) or introduce artificial delay/timeouts in your wrapper to see retries kick in.

Watch your logs to confirm:

- Retries run only on transient errors.
- The final response to the client is still clean and user-friendly.

Step 4: Capture Evidence for Output Screenshot

Take a screenshot showing:

- Postman response (with your meta section or similar).
- OR your console logs with retries and token usage visible.

Expected Outcome

A tested middleware that behaves predictably and surfaces clear logging.

At least one screenshot showing successful retry or cost logging in action.

Tips

If you can't force real rate limits, you can simulate an error in your code (e.g., throw an error on the first attempt) just to confirm retry logic works.

Use consistent sample messages so it's easier to see caching and retry patterns.

Compile Your Findings

1 Middleware Script

→ **Deliverable:** Core backend implementation wrapping OpenAI API calls.

→ **Include:**

→ Source code for your FastAPI or Node.js service.

→ The retry logic, exponential backoff, and cache implementation.

→ Any comments that clarify design choices.

2

Optimization & Trade-offs Explanation

Deliverable: Short explanation file (or rich-text answer).

Include:

Why you chose your specific retry strategy and backoff parameters.

How caching reduces both cost and rate-limit pressure.

Any trade-offs you considered (e.g., stale cache risk, added complexity).

3

Output Screenshot

Deliverable: Visual evidence of your middleware in action.

Include:

Screenshot of Postman response showing your stable JSON structure and metadata.

OR screenshot of logs showing retries and token usage for a request.

Summary

This activity provides hands-on experience with **backend reliability for LLM apps**. Instead of calling the OpenAI API directly and hoping for the best, you've built a middleware layer that:

- Handles timeouts and transient errors gracefully with structured retries and exponential backoff.
- Uses caching to reduce redundant calls, lower costs, and ease rate-limit pressure.
- Logs token usage and retry counts so performance and cost are visible—not guesswork.

Key takeaways

- Reliable LLM apps don't just depend on model quality; they depend on robust backend design.
- Middleware is where you absorb chaos—errors, rate limits, network quirks—so users see a calm, predictable experience.
- Simple metrics (tokens, retries, cache hits) give you the observability you need to scale with confidence.

Resources

Primary Materials:

- Course examples for FastAPI/Node.js wrappers from Module 2 videos.

Reference Documentation:

- OpenAI API docs (rate limits, usage fields, error codes).
- FastAPI or Express/Nest documentation (for HTTP routing, middleware patterns).

Additional Resources (Optional):

- Articles or snippets on exponential backoff patterns.
- Caching libraries or Redis examples, if you want to extend beyond in-memory cache.

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.